

CS205 C/C++ Programming - Project 3: Improved Matrix Multiplication in C

Name: 殷海洲(Yin Haizhou)

SID: 12210226

Part 1 - Analysis

Requirement

1. Only float is needed since it is the most commonly used data type in deep learning.
2. Implement a function `matmul_plain()` in a straightforward way using several loops as the benchmark.
3. Implement a function `matmul_improved()` using SIMD, OpenMP and other technologies to improve the speed. You can compare it with `matmul_plain()`.
4. Test the performance using 16x16, 128x128, 1Kx1K, 8Kx8K and 64Kx64K matrices. You can generate some matrices with random values to test your function.
5. Compare your implementation with the matrix multiplication in OpenBLAS(<https://www.openblas.net/>). The results of your implementation should be the same or very similar to those of OpenBLAS. Can your implementation be as fast as OpenBLAS?
6. [Optional] Test your program on X86 and ARM platforms, and describe the differences.

Based on requirements, a function that realizes matrix multiplication will be applied as the one has been developed in Project 2 Simple Matrix Multiplication.

Then, the task is to improve it using SIMD, OpenMP and etc. to accelerate the process of calculation while keep its accuracy. Also, a function including matrix

multiplication using Openblas library is also required.

In the end, we apply all these functions calculate matrix multiplication and compare the speed of them.

Program Design

How I design my program according to requirements

Initially, a matrix structure will be defined with three elements which are the numbers of rows and columns and the data pointer pointing to the data of a matrix.

Simultaneously, a structure function `create()` is defined in which according to the input row and column numbers, a matrix will be created and data will be allocated. However, this function also rejects invalid inputs of numbers to make sure that matrices are not too big for the computer.

Then, to define a function is easy, but it is essential to validate the parameter before using it. As for matrix multiplication, it is a structure function in which all parameters and return value are structure `Matrix` and pass by pointers, so they must not be `NULL` if used. An alarm needs to be called to cease the program and to report problems.

As for `matmul_plain()` , it consists of three normal loops to map all elements and return a result which is 100% accurate.

As for `matmul_improved()` , three functions were devised including one simply using SIMD `AVX2` , one using both SIMD `AVX2` and `OpenMP` and one ~~showing intelligence of the author~~ applying ***partitioned matrix algorithm*** which is cache-friendly so that speeds up the process.

Moreover, a function using OpenBLAS Library is devised using a function `sgemm()` to calculate matrix multiplication.

Referring to counting time, there are two fundamental ways. One is `clock()` in `<time.h>`, and the other is `omp_get_wtime()` in `<omp.h>`. Both functions record the current time of systems and returned a double type number, but `clock()` fails to record an accurate time of a program using multiple threads. In a word, `omp_get_wtime()` will be used while there are many threads working together, while `clock()` will be used if only one thread is working.

Finally, all functions are applied so that we could measure time cost of each function. We manually change the size of matrix from 16x16, 128x128, 1kx1k, 8kx8k, 64kx64k and we could get the result.

Part 2 - Code

In order to make comparison with speed of different functions, I "comment" some parts to make them would not operate when I do not need them. This version of the code is a comparison between `matmul_plain()` and `matmul_openblas`.

2.1 Process in Advance

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cblas.h>

#ifdef WITH_AVX2
#include <immintrin.h>
#endif

#ifdef _OPENMP
#include <omp.h>
#endif
```

2.2 Define a Structure

Following tips, a matrix structure is defined.

```
typedef struct Matrix{
    size_t rows;
    size_t cols;
    float *data;
}Matrix;
```

2.3 Create a Matrix

Although this project only needs square matrices, a general matrix creator is used since that it is believed our work can be still used some time when matrices are no longer squares.

```
Matrix * create(size_t rows, size_t cols)
{
    if(cols>0&&cols<100000&&rows>0&&rows<100000)
    {
        Matrix *matrix = (Matrix *)malloc(sizeof(Matrix));
        matrix->cols = cols;
        matrix->rows = rows;
        matrix->data = (float*)malloc(matrix->rows * matrix->cols * sizeof(float));
        return matrix;
    }
    else
    {
        printf("The size of matrix is invalid.\n");
        printf("Please make sure that it is positive and cannot be too big.\n");
        printf("The numbers of columns and rows should be from 1 to 10000\n");
        return NULL;
    }
}
```

2.4 Ordinary Matrix Multiplication

```
Matrix * matmul_plain(Matrix *A, Matrix *B)
{
    clock_t start,end;
    double time;
    if (A == NULL || B == NULL || A->cols != B->rows)
    {
        //check if A and B are valid
        printf("You have to make your calculation valid\n");
        return NULL;
    }
    else
    {
        Matrix *C = (Matrix *)malloc(sizeof(Matrix));
        //define a returned matrix C and allocate memory for it, in which there are two 4-byte
        C->rows = A->rows;
        C->cols = B->cols;
        C->data = (float *)malloc(C->rows * C->cols * sizeof(float));
        //start counting after preparation
        start = clock();
        for(int i = 0; i < C->cols; i++)
        {
            for(int j = 0; j < C->rows; j++)
            {
                C->data[j * C->cols + i] = 0;
                //initialize elements of C
                for(int k = 0; k < A->cols; k++)
                    C->data[j * C->cols + i] += A->data[j * A->cols + k] * B->data[k * B->cols + i];
            }
            //map all rows in C to multiply a column in B
        }
        end = clock();
        time = ((double) (end - start)) / CLOCKS_PER_SEC;
        printf("It costs %lf second.\n",time);
        return C;
    }
}
```

2.5 Improved Matrix Multiplication

```
Matrix * matmul_improved(Matrix *A, Matrix *B)
{
    #ifdef WITH_AVX2
        clock_t start,end;
        double time;
        if (A == NULL || B == NULL || A->cols != B->rows)
        {
            printf("You have to make the number of columns of A equal the number of rows of B\n");
            return NULL;
        }
        else if (A->cols % 8 != 0 || B->rows % 8 != 0)
        {
            printf("The number of columns of A should be a multiple of 8\n");
            printf("The number of rows of B should also be a multiple of 8\n");
            printf("AVX2 provides register for 256 bits so please make sure your matrices fit\n");
            return NULL;
        }
        else
        {
            Matrix* C = create(A->rows, B->cols);
            //transpose factor matrix B
            Matrix* transposed_B = create(B->cols, B->rows);
            start = clock();
            for (int i = 0; i < B->rows; i++)
            {
                for (int j = 0; j < B->cols; j++)
                    transposed_B->data[j * B->rows + i] = B->data[i * B->cols + j];
            }
            // Perform matrix multiplication using AVX2
            __m256 a,b,c;
            float c_result = 0.0f;
            float sum[8] = {0}; //once initialize, later be covered
            for(int i = 0; i < C->cols; i++)
            {
                for(int j = 0; j < C->rows; j++)
                {
                    c = _mm256_setzero_ps(); //similar with sum array
```

```

        c_result = 0.0f;
        //setzero
        for(int k = 0; k < A->cols; k+=8)
        {
            a = _mm256_loadu_ps(A->data + k + j*A->cols);
            b = _mm256_loadu_ps(transposed_B->data + i*transposed_B->cols + k);//mul
            c = _mm256_mul_ps(a,b);
            _mm256_storeu_ps(sum,c);
            for (int l = 0; l<8;l++)
                c_result += sum[l];
        }
        C->data[j * C->cols + i] = c_result;//k loop calculates an entry of c[i][j]
    }//map all rows in A to multiply a column in B
}

end = clock();
time = ((double) (end - start)) / CLOCKS_PER_SEC;
printf("It costs %lf second.\n",time);
return C;
}

#else

    printf("Error!AVX2 is not supported.\n");
    return NULL;

#endif
}

```

```

Matrix * matmul_improved_OPMP(Matrix *A, Matrix *B)
{
    #ifdef WITH_AVX2
        double start,end,time;//When multiple threads are used, the time is not accurate using
        if (A == NULL || B == NULL || A->cols != B->rows)
        {
            printf("You have to make the number of columns of A equal the number of rows of B\n");
            printf("You have to make sure your calculation is valid\n");
            return NULL;
        }
        else if (A->cols % 8 != 0 || B->rows % 8 != 0)
        {

```

```

printf("The number of columns of A should be a multiple of 8\n");
printf("The number of rows of B should also be a multiple of 8\n");
printf("AVX2 provides register for 256 bits so please make sure your matrices fit\n");
return NULL;
}
else
{
    Matrix* C = create(A->rows, B->cols);
    //transpose factor matrix B
    Matrix* transposed_B = create(B->cols, B->rows);
    for (int i = 0; i < B->rows; i++)
    {
        for (int j = 0; j < B->cols; j++)
            transposed_B->data[j * B->rows + i] = B->data[i * B->cols + j];
    }
    __m256 a,b,c;
    float c_result = 0.0f;
    float sum[8] = {0}; //once initialize, later be covered
    start = omp_get_wtime();
    #pragma omp parallel for
    for(int i = 0; i < C->cols; i++)
    {
        for(int j = 0; j < C->rows; j++)
        {
            c = _mm256_setzero_ps(); //similar with sum array
            c_result = 0.0f; //setzero
            for(int k = 0; k < A->cols; k+=8)
            {
                a = _mm256_loadu_ps(A->data + k + j*A->cols);
                b = _mm256_loadu_ps(transposed_B->data + i*transposed_B->cols + k); //mul
                c = _mm256_mul_ps(a,b);
                _mm256_storeu_ps(sum,c);
                for (int l = 0; l<8;l++)
                    c_result += sum[l];
            }
            C->data[j * C->cols + i] = c_result; //k loop calculates an entry of c[i][j]
        } //map all rows in A to multiply a column in B
    }
}

```



```

        end = omp_get_wtime();
        time = end - start;
        printf("It costs %lf second.\n",time);
        return C;
    }
#else
    printf("Error!AVX2 is not supported.\n");
    return NULL;
#endif
}

```

The improved matrix multiplication use SIMD and multi-thread to speed up the process. SIMD calculate 8-length float vector dot product in one go and OpenMP use multiple threads to calculate at the same time.

2.6 Partitioned Matrix Multiplication

The mathematics rules for partitioned matrix multiplication is that for matrix A($m \times p$) and B($p \times n$), we divide A into small parts(8×8).

Then we multiply blocks as if they are numbers.

This reason I choose 8 is that 8 fits from sizes of 16 to 64k. However, it could also be anticipated that with the size of matrices increasing, it would gradually slow down just like how the previous function does.

In this function, two matrices are passed by pointers and a result matrix will be created. Then A and B are calculated in which results are written in matrix C and returned as a pointer to result matrix.

This function works in a rapid way but less accurate compared with other functions.

```

Matrix * matmul_improved_divided(Matrix *A, Matrix *B)
{
    if (A == NULL || B == NULL || A->cols != B->rows || A->cols%8!=0 || A->rows%8!=0 || B->cols%8!=0 || B->rows%8!=0)
    {
        printf("You have to make the number of columns of A equal the number of rows of B\n");
        printf("The lateral length must be multiples of eight\n");
        printf("You have to make sure your calculation is valid\n");
        return NULL;
    }

    Matrix * C = create(A->rows,B->cols);
    for (int i = 0;i<C->rows;i++)
    {
        for(int j = 0;j<C->cols;j++)
            C->data[i*C->cols + j] = 0;
    }//initialize C

    //A has i*k blocks, while B has k*j blocks
    //calculate C in blocks, while consequently C has i*j blocks, The ij block equals dot product of A and B
    double start,end,time;
    start = omp_get_wtime();
    #pragma omp parallel for
    for (int ci = 0; ci < C->rows; ci += 8) //iterate with C rows
    {
        for (int cj = 0; cj < C->cols; cj += 8) //iterate with C columns
        {
            for (int ak = 0; ak < A->cols; ak += 8) //iterate with A columns, doing dot product of A and B
            {
                for (int a = 0; a < 8; a++) //for each block, we apply similar algorithm as normal
                {
                    for (int b = 0; b < 8; b++)
                    {
                        float sum = 0;
                        for (int c = 0; c < 8; c++)
                            sum += A->data[(ci + a) * A->cols + ak + c] * B->data[(ak + c) * B->cols + cj + b];
                        C->data[(ci + a) * C->cols + cj + b] += sum;
                    }
                }
            }
        }
    }
    end = omp_get_wtime();
    time = end - start;
    printf("Time taken: %f\n", time);
    return C;
}

```

```

        }
    }
}
end = omp_get_wtime();
time = end - start;
printf("It costs %lf second.\n",time);
return C;//Be careful of low accuracy
}

```

2.7 OpenBLAS Matrix Multiplication

To regulate my functions and to count times, a function was created for OpenBLAS matrix multiplication.

```

void matmul_openblas(Matrix *A, Matrix *B, Matrix *C)
{
    if (A == NULL || B == NULL || C == NULL || A->cols != B->rows || C->rows != A->rows || C->cols != B->cols)
    {
        printf("You have regulate sizes of A, B, C\n");
        if(C != NULL)
        {
            free(C->data);
            C->data = NULL;
        }
        return;
    }
    double time,start,end;
    start = omp_get_wtime();//openblas also uses multiple threads
    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, A->rows, B->cols, A->cols, 1.0, A->data, B->data, 0.0, C->data);
    //The meaning of parameters refer to the following:
    //row priority, no transpose, no transpose, rows of result matrix, columns of result matrix, scalar multiplier,
    //gain, data of A, columns of A, data of B, columns of B, extra_addition, address of C, column stride of C
    end = omp_get_wtime();
    time = end - start;
    printf("It costs %lf second.\n",time);
}

```

```
    return;
}
```

2.8 Implement Main Function

```
int main()
{
    int length;
    int ret = scanf("%d",&length);
    size_t size = (size_t)length;

    Matrix * matrix1 = create(size,size);
    Matrix * matrix2 = create(size,size);

    Matrix * matrix_result1 = matmul_plain(matrix1,matrix2);
    // Matrix * matrix_result2 = matmul_improved(matrix1,matrix2);
    // Matrix * matrix_result2 = matmul_improved_OPMP(matrix1,matrix2);
    //Matrix * matrix_result2 = matmul_improved_divided(matrix1,matrix2);

    Matrix * matrix_result2 = create(size,size);
    matmul_openblas(matrix1,matrix2,matrix_result2);

    if (matrix_result1 != NULL&&matrix_result2 != NULL)
        printf("Matrices are successfully mutiplied.\n");
    else
        printf("Functions failed to create matrices or multiply.\n");

    Matrix *ptr[4] = {matrix1,matrix2,matrix_result1,matrix_result2};

    for(int i = 0;i < 4;i++)
    {
        if ((ptr[i]) != NULL)
        {
            if(ptr[i]->data != NULL)
                free(ptr[i]->data);

            free(ptr[i]);

            printf("Matrix info freed.\n");
        }
    }
```

```
}  
  
//make sure that memory freed  
  
}
```

In this code, only `matmul_plain()` and `matmul_openblas` are invoked and the others are not selected.

To compile it, following commands are required.

```
gcc matrix.c -O3 -fopenmp -DWITH_AVX2 -mavx2 -lopenblas -o openblas
```

All require O3, even the plain one, or otherwise a lot of time will be spent.

Part 3 - Result & Verification

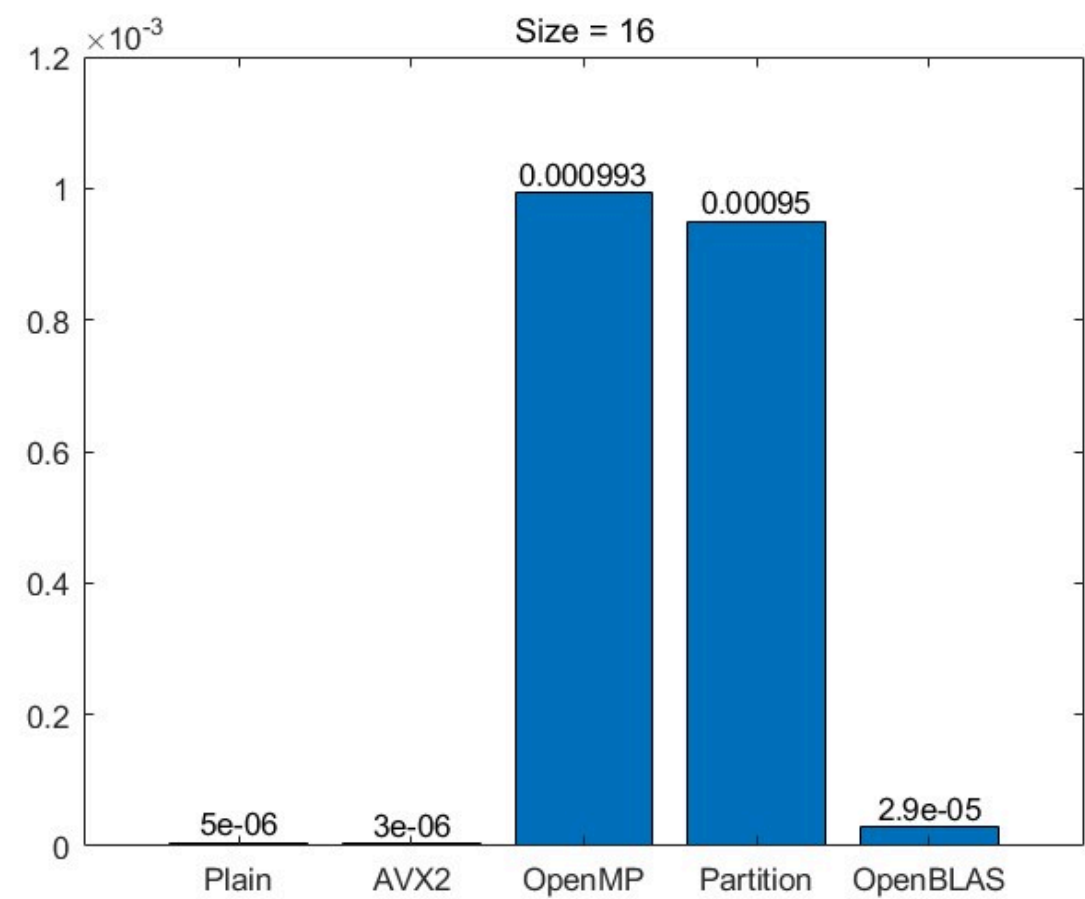
3.1. Time cost comparison

3.1.1 Comparison on PC

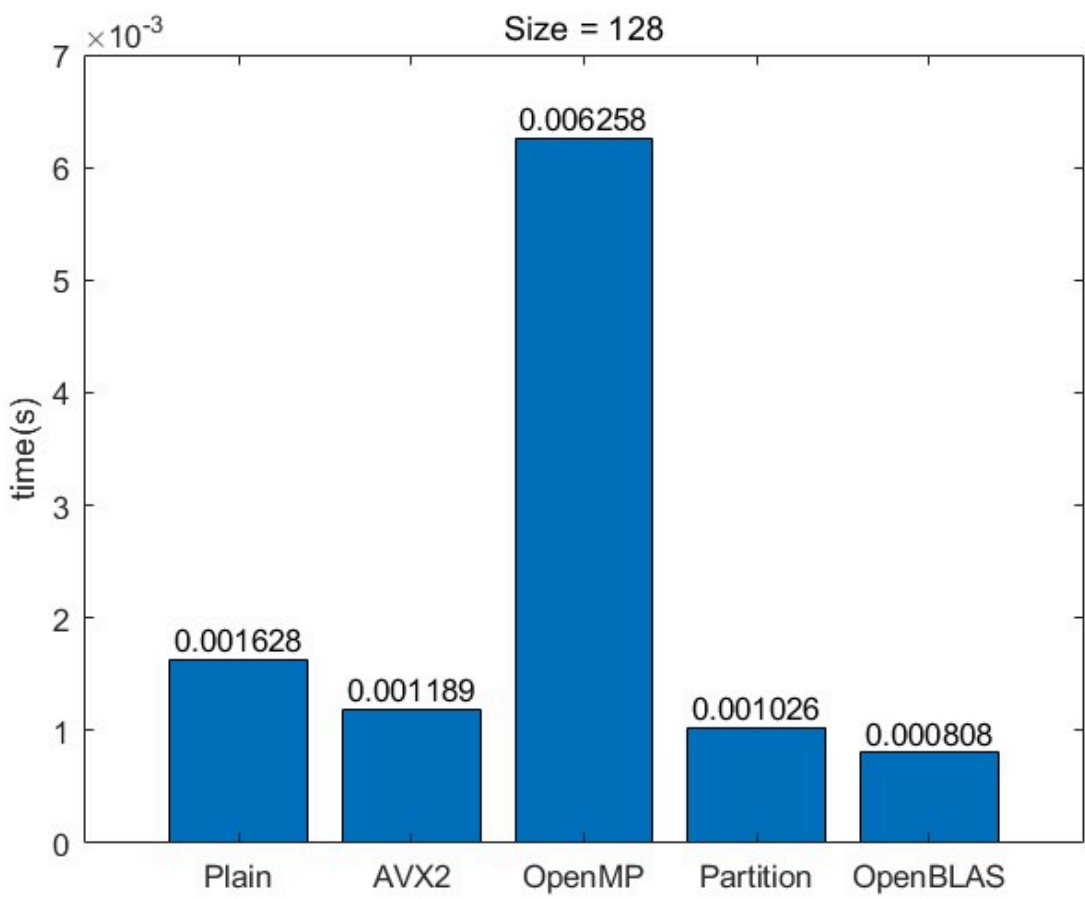
Running the code on my own laptop, we got a series of results with size = 16, 128, 1k and 2k. The result is showed below.

My laptop failed to give a result when the size of matrix is over 8000.

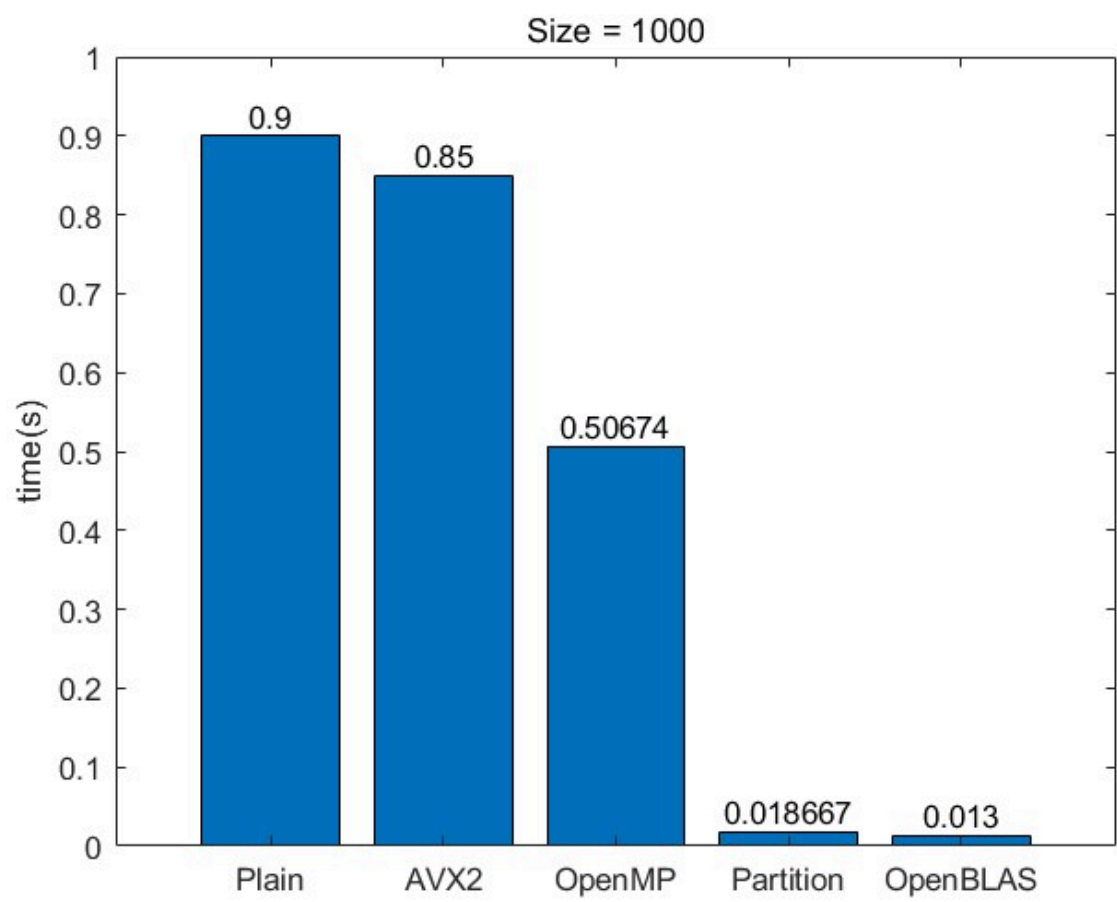
SIZE = 16



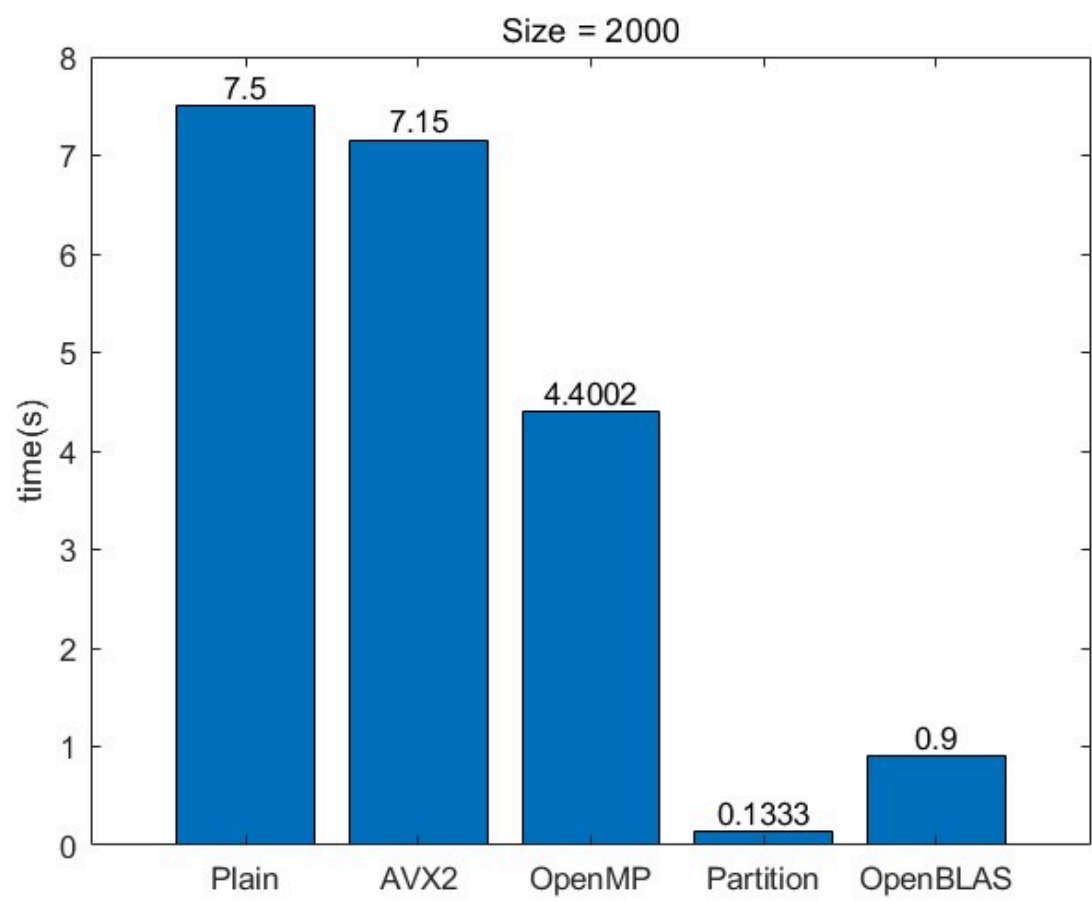
SIZE = 128



SIZE = 1000



SIZE = 2000



From figures, it indicates that `matmul_plain` and `matmul_improved` which used AVX2 is fastest when size is small. This is probably because they are one-threaded and do not requires extra expense as those multi-threaded.

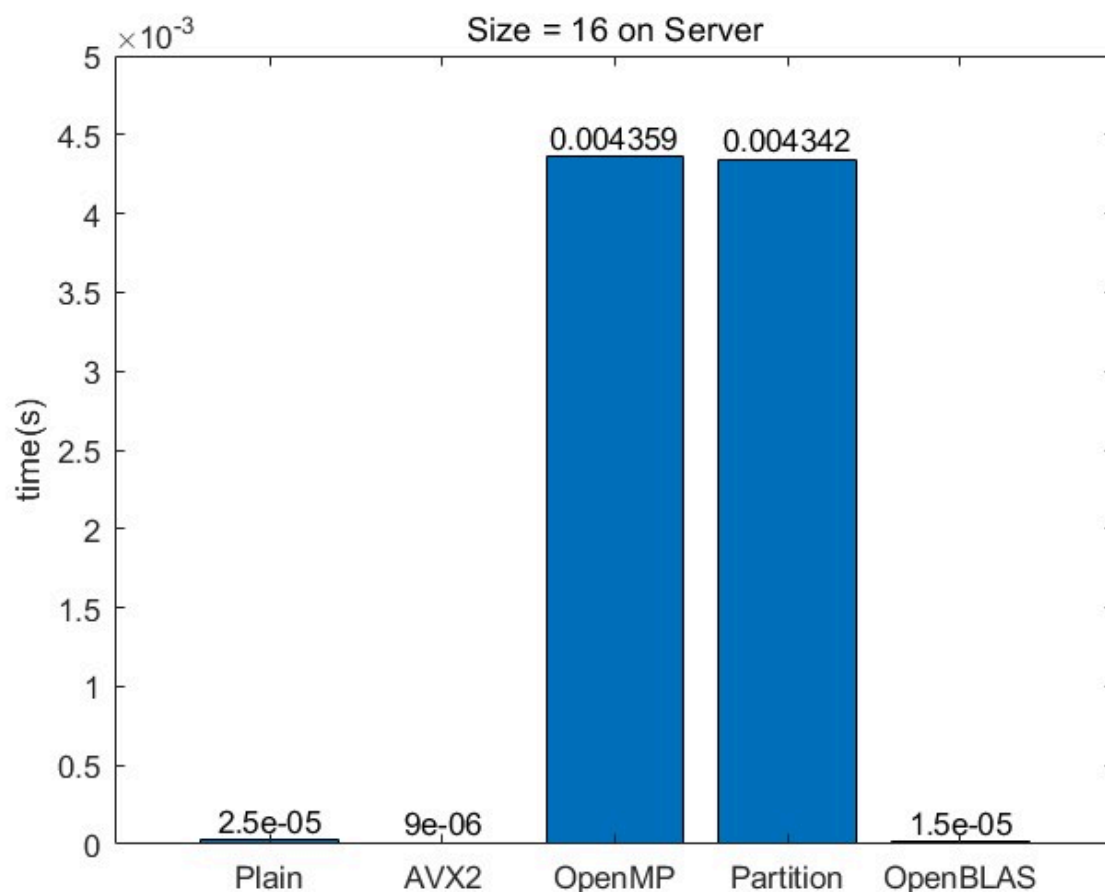
When the size grows to 1000, one-threaded functions slow down and multi-threaded preceded. Among them, partitioned matrix multiplication won when SIZE = 2000 and this is probably because it is not accurate as OpenBLAS.

3.1.2 Comparison on Server

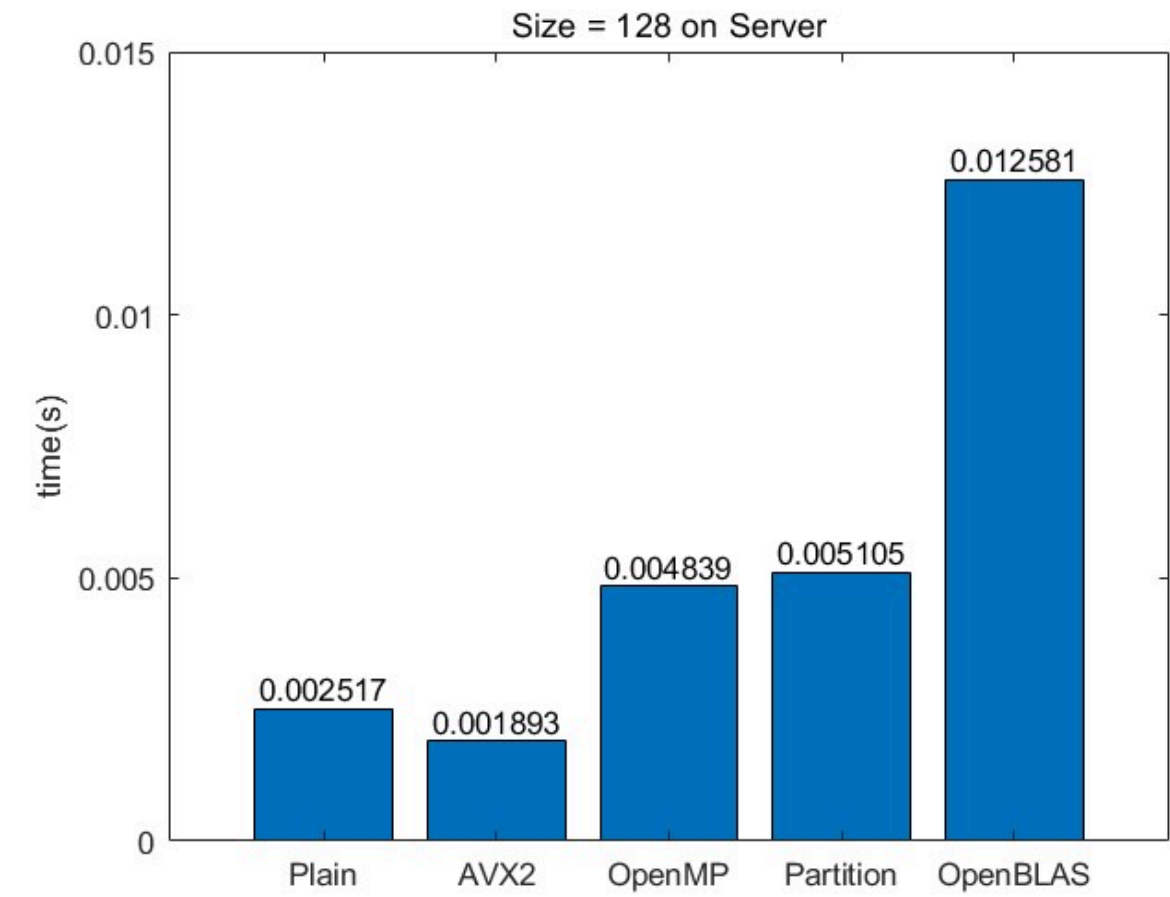
Running the code on my own laptop, we got a series of results with size = 16, 128, 1k, 8k and 16k, in which only partitioned matrix multiplication and openblas support the last two sizes. The result is showed below.

The result varies a lot so I could only observe and deduce.

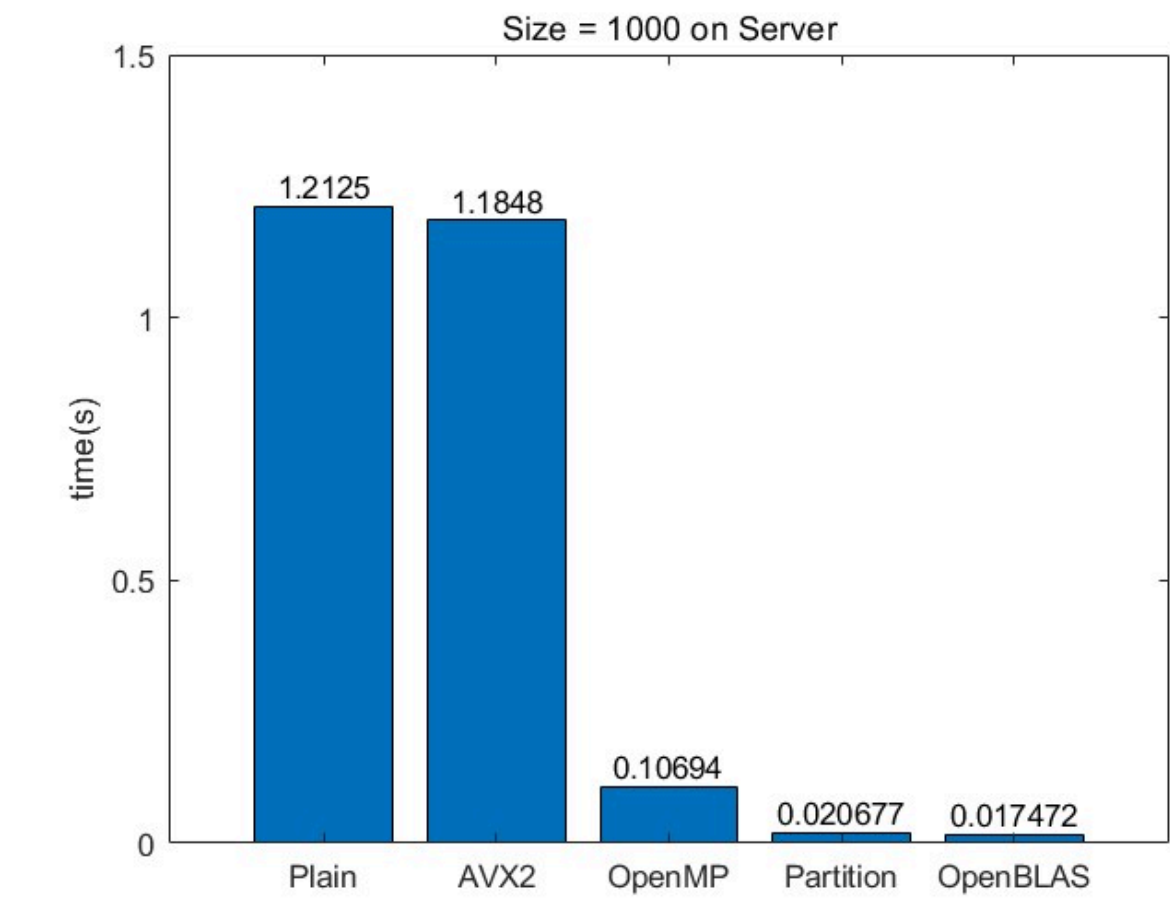
SIZE = 16



SIZE = 128

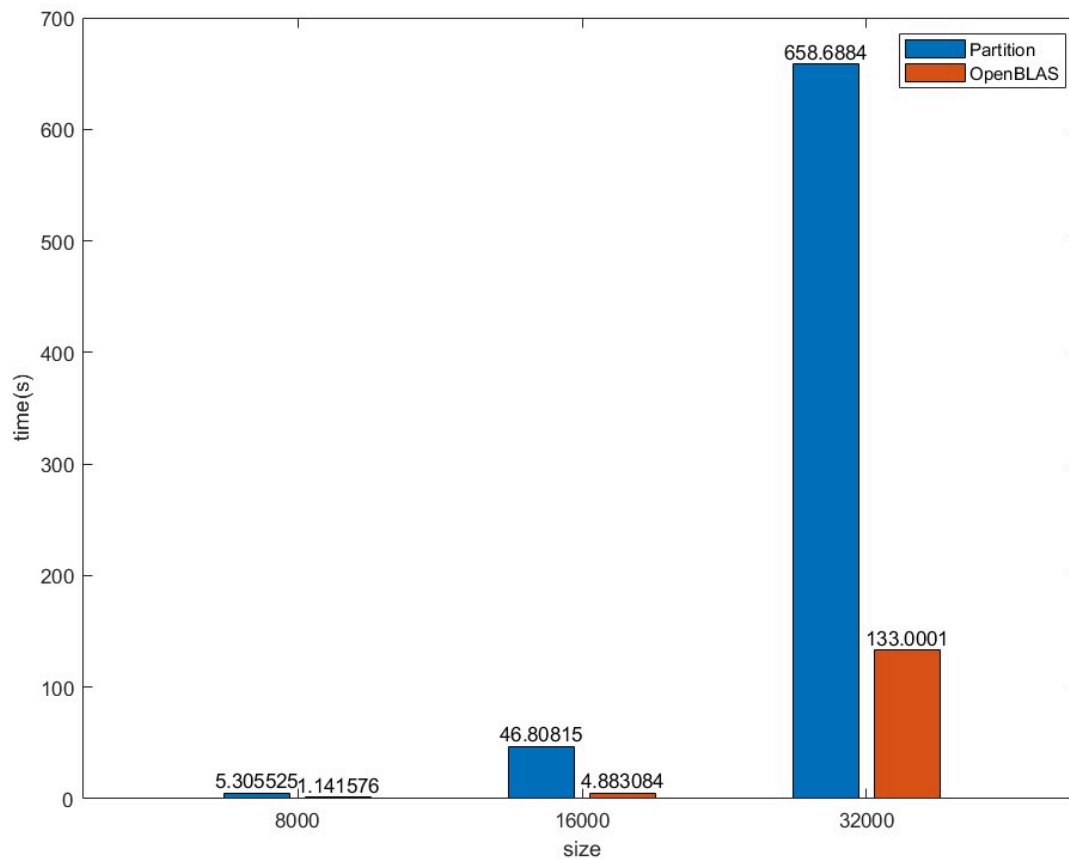


SIZE = 1000



SIZE = 8000, 16000, 32000

Set a valid time = 600s, only partitioned multiplication and OpenBLAS successfully ran.



SIZE = 64000

When Size gets to 64000, all personal methods failed to give a result before being ceased but OpenBLAS could still run to calculate matrix multiplication. The result will be around 300s.

As a conclusion, Openblas is fast when size is extremely large, while plain matrix multiplication and partitioned matrix multiplication also showed their prior ranges. To explain this, plain algorithm are simple so it is fast when data is little and partitioned algorithm operates well if the size is suitable which means balancing expense on loading and data size.

3.2. Accuracy Verification

In order to check the accuracy of the program, we need some stress test program to verify. Here, we introduce two test programs.

3.2.1 Ambiguous Stress Test

This program compares result matrices from different multiplication functions entry by entry. A sufficiently small number is set as an acceptable error. Then, this program will map all entries and verify their error is smaller than the set value.

Part of the first program is as follows.

```
bool compare_matrices(Matrix* A, Matrix* B)
{
    if (A->rows != B->rows || A->cols != B->cols)
    {
        printf("size!\n");
        return false; // Matrices have different dimensions
    }
    float epsilon = A->cols * 1e-5f; // error
    for (int i = 0; i < A->rows * A->cols; i++)
    {
        if (fabs(A->data[i] - B->data[i]) > epsilon)
        {
            printf("Entry! the %d\n", i);
            printf("A:%f\n", A->data[i]);
            printf("B:%f\n", B->data[i]);
            return false; // Matrices are not equal
        }
    }
    return true; // Matrices are equal
}
```

Only the most significant function of this program is provided but it clarifies well how it works. You may notice that the error `epsilon` is set as a strange form deliberately and this will be explained later.

3.2.2 Visualized Stress Test

This program prints out the result of a intentionally set matrix A and an identity matrix B, it helps guarantee that one function is originally correct. If not, it is possible to show the problem as well.

Part of the second program is as follows.

```
void print_matrix(Matrix* matrix)
{
    for (int i = 0; i < matrix->rows; i++)
    {
        for (int j = 0; j < matrix->cols; j++)
            printf("%.2f ", matrix->data[i * matrix->cols + j]);
        printf("\n");
    }
}

int main()
{
    // Create a matrix
    Matrix* A = create(8, 8);
    printf("Matrix A:\n");
    print_matrix(A);

    // Create an identity matrix
    Matrix* I = create(8, 8);
    for (int i = 0; i < 8; i++)
    {
        for (int j = 0; j < 8; j++)
            I->data[i * 8 + j] = (i == j) ? 1.0f : 0.0f;
    }
}
```

```

Matrix* result = matmul_improved(A, I);
if (result != NULL)
{
    printf("\nResult of A * I:\n");
    print_matrix(result);
}

free(A->data);
free(A);
free(I->data);
free(I);
if (result != NULL)
{
    free(result->data);
    free(result);
}
return 0;
}

```

It is initially designed to check why the early type of `matmul_improved` had failed. By Using it, many interesting problems and discoverings became no more invisible.

Part 4 - Difficulty & Solution

1. The very first difficulty is how to make sure that matrix multiplication is correct if there is no OpenBLAS at a very early stage. As it is mentioned in the last part, stress tests were done to ensure a accurate result is returned.
2. When using AVX2 as a SIMD to speed up the program, we introduce a new type `__m256` to store 8 float numbers and to calculate. It resulted in a wrong result at first, and I spent a lot of time to check my code with multiple ways, while AI didn't find out the problem at all. Finally, I realized on my own that it was a result of linear storage with row prior. `__m256` loads 8 numbers automatically and

continuously with one address, so it loaded the row entries of B instead of column entries, which is supposed to be loaded. To solve it, a transpose operation was invoked before matrix multiplication, so the order of loading will be correct.

3. Though partitioned matrix multiplication is super fast beyond my expectation, it actually has some problems with accuracy compared with others. With the size increasing, its accuracy falls as well. To pass the stress test, permission of less accuracy was introduced so the error was set as a varying value as above.
4. Partitioned matrix multiplication are originally designed with SIMD. Nevertheless, 8x8 blocks appear to be so tiny that it results in extra expense using SIMD and inaccurate results. Therefore, fundamental datatypes are used instead.
5. Pointers are massively used in this project so problems occur and trouble. Careful inspection was adopted on very use of pointer to avoid wild and dangling pointers, memory leak and segmentation fault. Every parameter passed into function must be carefully restrained.
6. Data analysis is also a big problem, since multi-thread program all were not stable to give similar results, so we could only regard the center of oscillation as their time costs. Some sizes that are not supported needed also to be estimated.
7. Many times the program failed to run on the server when size = 64k. a entire day was spent on fixing this problem. It turns out to be that int was used instead of size_t datatype and resulted in serious segmentation fault. It would be fine after switching int to size_t.

Part 5 - Appendix

experience, comments and reference

5.1 Experience and Comments

I thought initially it was supposed to be an easy project but I found that it ended up difficult and detailed. There are many things that need to be cared for. For instance, to prove the AVX2 matrix multiplication would return a correct answer. I didn't realize I should transpose matrix for storage until I printed all of entries out.

Meanwhile, I did a lot of research about matrix multiplication algorithms and I was attracted by partitioned matrix multiplication and Winograd Algorithm. You may notice that there is no mention of Winograd Algorithm in the body part because I found that the function I composed is even slower than the plain one.

Also, it is not easy to apply partitioned matrix multiplication because it is ambiguous to locate every entry. Despite its low accuracy, it surprised me with its speed compared to other functions except OpenBLAS.

When I tried to run a 64k matrix multiplication, I met a lot of segmentation fault and spent a large amount of time to figure out the problem while 32k ran successfully. Actually, it turned out to be a result of overflow which has been warned and emphasized for many times on lecture 2 and I felt quite embarrassed.

When I tried to run my program, there are many users on it using it and what I could use is limited. I would consider starting doing project in advance next time.

Although many times I expected my partitioned matrix multiplication would beat OpenBLAS, it ended up with OpenBLAS winning.

I'm interested now in how OpenBLAS made this. I would like to spend my spare time during vacation to figure out it.

Reference

Github [OpenBLAS](#)

CSDN Blog [c++实现矩阵乘法 and 分块矩阵乘法](#)

CSDN Blog [Winograd 算法原理](#)